

Lab 07: Verifying Software Integrity (SHA256 & GPG)-Exercise Report

1. Inspect the script

```
(kali㉿kali)-[~/CS-Vaje/lab07]
$ cat > install.sh << 'EOF'
#!/bin/bash
mkdir -p src
cat > run.sh << 'INNER'
#!/bin/bash
echo "Hello"
INNER
mv run.sh src/
chmod +x src/run.sh
./src/run.sh
EOF
chmod +x install.sh
```

```
(kali㉿kali)-[~/CS-Vaje/lab07]
$ cat install.sh
#!/bin/bash
mkdir -p src
cat > run.sh << 'INNER'
#!/bin/bash
echo "Hello"
INNER
mv run.sh src/
chmod +x src/run.sh
./src/run.sh
```

```
(kali㉿kali)-[~/CS-Vaje/lab07]
$ sudo apt install gnupg coreutils
[sudo] password for kali:
gnupg is already the newest version (2.4.9-7).
gnupg set to manually installed.
coreutils is already the newest version (9.10-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 827
```

Q What does this script do ?

In this exercise, I simulated a simple software installation process. I created a src directory and a run.sh script that prints **Hello**. Then, I moved the script into the src directory and executed it using src/run.sh. This helped me understand how files and scripts can be organized and executed in Linux.

2. Generate SHA256 Cheksum

```
(kali㉿kali)-[~/CS-Vaje/lab07]
$ sha256sum install.sh > install.sh.sha256

(kali㉿kali)-[~/CS-Vaje/lab07]
$ cat install.sh.sha256
13679fcef8039ec3c8b33cc7fd3ebba97466e49c9f0889f917227f56395c27ed  install.sh
```

3. Verified file Integrity

```
(kali㉿kali)-[~/CS-Vaje/lab07]
$ sha256sum -c install.sh.sha256
install.sh: OK
```

4. Gerenerated GPG key full

```
--$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only)
 (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
  0 = key does not expire
<n> = key expires in n days
<n>w = key expires in n weeks
<n>m = key expires in n months
<n>y = key expires in n years
Key is valid for? (0) 1
Key expires at Thu 27 Aug 2026 05:02:07 PM EDT
Is this correct? (y/N) n
Key is valid for? (0) 1y
Key expires at Thu 26 Aug 2027 05:02:17 PM EDT
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: mallory
Email address: malory @ext.vom
Comment:
You selected this USER-ID:
  "mallory <malory @ext.vom>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? o
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/68833EE47B3A3A3B464718DEFBA2D68981E23ED0.rev'
public and secret key created and signed.
```

5. Signed the Checksum File

```

(kali@kali)-[~/CS-Vaje/lab07]
$ gpg --list-secret-keys --keyid-format LONG
gpg --local-user KEYID --detach-sign install.sh.sha256
gpg: checking the trustdb
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: depth: 0 valid: 4 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 4u
gpg: next trustdb check due at 2027-08-18
/home/kali/.gnupg/pubring.kbx

sec   rsa4096/074137CAF84D06A1 2026-08-18 [SC] [expires: 2027-08-18]
uid    [ultimate] vk (vk-vk) <vk@ert.com>
ssb    rsa4096/55B9B021FC9BE99E 2026-08-18 [E] [expires: 2027-08-18]

sec   rsa4096/56FC8BB9678CF841 2026-08-18 [SC] [expires: 2027-08-18]
uid    [ultimate] dona (Dona) <dona@example.com>
ssb    rsa4096/C97608F1EFBCA438 2026-08-18 [E] [expires: 2027-08-18]

sec   rsa4096/9C8857BAD8287CD0 2026-08-26 [SC] [expires: 2027-08-26]
uid    [ultimate] Vk <vk@gmail.com>
ssb    rsa4096/C56E0E5B9AD5443C 2026-08-26 [E] [expires: 2027-08-26]

sec   rsa4096/FBA2D68981E23ED0 2026-08-26 [SC] [expires: 2027-08-26]
uid    [ultimate] mallory <malory@ext.vom>
ssb    rsa4096/1683C36E330F4F0D 2026-08-26 [E] [expires: 2027-08-26]

gpg: skipped "KEYID": No secret key
gpg: signing failed: No secret key

```

6. Verified the Signature

```

(kali@kali)-[~/CS-Vaje/lab07]
$ gpg --verify install.sh.sha256.sig install.sh.sha256
gpg: Signature made Wed 26 Aug 2026 05:04:12 PM EDT
gpg: using RSA key 6B8E1AFC71CED4ACB31E8AFC074137CAF84D06A1
gpg: Good signature from "vk (vk-vk) <vk@ert.com>" [ultimate]

```

7. Simulated a Tampering Attack

```

(kali@kali)-[~/CS-Vaje/lab07]
$ echo "echo MALICIOUS CODE EXECUTED" >> install.sh

(kali@kali)-[~/CS-Vaje/lab07]
$ sha256sum -c install.sh.sha256
install.sh: FAILED
sha256sum: WARNING: 1 computed checksum did NOT match

```

Questions & Answers

Why should you never execute scripts without inspection?

Before running any script, I learned that it is important to read and understand it first. A script runs with the user's permissions and can make changes to the system. A malicious script could delete files, steal personal information, install malware, change system settings, or connect to unknown servers. Therefore, I should always check a script before executing it, especially if it comes from an untrusted source.

What does SHA256 guarantee?

In this exercise, I learned that **SHA256** is used to check the integrity of a file. If even a small part of the file is changed, the SHA256 checksum will be completely different. This helps detect whether a file has been modified. However, SHA256 cannot confirm who created the file or whether the source of the file is trustworthy.

What does GPG guarantee?

GPG guarantees both authenticity and integrity. A valid signature shows that the signer had the corresponding private key when signing the file. The signer's identity can be checked using their key fingerprint, and the signature also confirms that the data has not been changed since it was signed.

Can SHA256 prove the author of a file?

No. SHA256 is only used to check the integrity of a file and does not contain any identity information. Anyone can calculate a new SHA256 checksum for a modified file, so it cannot prove who originally created the file.

Why is signing the checksum important?

Signing the checksum provides extra protection. SHA256 confirms that the file has not changed, while the GPG signature connects the checksum to the signer's identity. An attacker would need the signer's private key to create a valid signature for a modified file.

What happened after the script was modified?

After adding the line `echo MALICIOUS CODE EXECUTED`, the SHA256 checksum changed and the `sha256sum -c` command reported **FAILED**. This detected that the script had been modified before it was executed.